

Documentazione Progetto Overseas Exchange

BASI DATI luglio 2026

- Luis Federico Soubie & Joseph Frigo

Autore: Joseph Frigo

Co-Autore: Luis Federico Soubie

Introduzione

Il progetto è suddiviso in 3 parti: **base dati**, **backend**, **frontend**, ognuno nelle rispettive cartelle /backend /frontend /database (o root del progetto /).

1. Backend

Il backend nella cartella /backend è scritto in **flask** 3.1.3 + **sql alchemy** 2.0.49, l'entry point dell'applicazione flask è il file app.py, le route principali fornite dall'API si trovano nei file *.py della cartella /backend/routes dove ogni file riporta funzionalità relative alle tabelle (e.g. **exams.py** per interfacciare con la tabella exams & mapped_exams, **application.py** per interfacciare con le applicazioni create dallo studente). I modelli del database vengono forniti dal file /backend/models.py, i quali consentono di utilizzare ORM per comunicare con le tabelle del database.

2. Database

Il database utilizzato è **PostgreSQL** 17.10, viene inizializzato in un container docker /Dockerfile, il suo contenuto [del database] viene popolato soltanto quando anche il container del backend va online, esso esegue il command "*flask db upgrade*" e popola il database con le informazioni fornite dai file migration che si trovano nella cartella /backend/migrations/versions/*.py

3. Frontend

Il frontend è scritto utilizzando il framework **Angular** 21.2.0, la struttura delle cartelle è mantenuta, le component principali si chiamano application-view, application-form, applications-list insieme ad altre componenti come il login, home e le componenti per la gestione riservata allo staff (il gestionale). Il backend del progetto è collegato al frontend grazie a delle funzioni intermedie (chiamati anche servizi) che astraggono l'API backend, queste funzioni si trovano nella cartella /frontend/src/app/api/*.ts, ognuno segue un formato ben specifico e ognuno riceve e restituisce delle interfacce [strutture dati] accurate di cosa si aspetta e cosa restituisce l'applicazione backend flask.

Funzionalità Principali

- Login Utente (per tutti gli utenti) + Logout Utente (utilizzando JWT token)

Applicazione

- Inserire nuove applicazioni (solo studente), l'utente è abilitato anche all'inserimento di più applicazioni in mod controllato (vedere sezione Ulteriori informazioni).

- Visualizzare la lista di applicazioni
 - vengono fornite in base all'utente: create dall'utente (per lo studente), associate al docente referente (per il docente), associate all'istituto (per lo staff)
 - ricerca delle applicazioni in base al nome dell'istituzione sending o host
 - filtrare le applicazioni in base allo stato
- Approvare o Rifiutare mappature di esami (solo docenti referenti)
- Approvare o Rifiutare documenti "learning agreement" o "transcript of records" (solo docenti referenti)
- Fornire Modifiche al learning agreement ed alla mappatura degli esami (solo studenti) durante "mobility ongoing" (è consentita una sola modifica in pending alla volta)
 - approvare o rifiutare le modifiche proposte (solo docente referente), nel caso in cui le modifiche vengono approvate tutti gli esami vengono cambiati e approvati, idem per il documento learning agreement (cambiato e approvato), se invece le modifiche vengono rifiutate si ripristinano le mappature degli esami precedente e il learning agreement precedente.
- Modifiche alla data di arrivo (inizio mobilità) e data di partenza (fine mobilità) quando lo stato dell'applicazione è in "mobility_ongoing"
- Inserire il voto e la data dello superamento esame (per lo studente, quando l'applicazione è in stato "exam_recognition")
- Approvare o Rifiutare I voti & data superamento esame delle mappature esami (solo docente referente)
- Chiudere [terminare] l'applicazione (solo staff)

Gestionale (solo staff)

- Inserire, modificare o eliminare Utenti (solo staff, e solo per utenti nella propria istituzione)
- Inserire ed Eliminare partnership con altre istituzioni (solo staff, e solo per il proprio istituto)
- Inserire ed Eliminare Esami (solo staff e solo per il proprio istituto)

Note per l'utilizzo

Per cominciare ad utilizzare l'applicazione deve entrare nella root del progetto (dove è presente il file docker-compose.yml ed eseguire `docker compose build --no-cache` seguito poi dal comando: **docker compose up**

Aprire un browser ed entrare l'url: <http://localhost:4200/home> per accedere al frontend (abbiamo deciso di tenere la porta 4200 per via del fatto che la porta 80 in alcuni sistemi operativi richiede un superuser per poter essere utilizzata).

Per cominciare ad utilizzare l'applicazione occorrono delle credenziali, ogni utente nella base dati ha la password impostata a "123", ecco alcuni utenti per poter entrare in autenticazione:

- marco.rossi@stud.uni.com (studente)
- laura.bianchi@prof.uni.com (docente referente)
- giuseppe.verdi@staff.uni.com (overseas staff)

ognuno degli utenti elencati appartiene all'Università Ca' Foscari, se vuole un elenco più completo esegua la seguente query nel database

postgresql://myuser:123@localhost:5432/overseas_d

```
select u.email, u.role as role, i.name
from users u
join institutions i on u.id_institution = i.id and i.id=1
where id_institution=1
order by u.id,u.role, i.name asc;
```

Approfondimento

Autenticazione

L'applicazione flask è equipaggiata con un'autenticazione e verifica di accesso, entralizzata nel file /backend/auth.py, questo file contiene 2 funzioni decorator usate come middleware, la prima è

`@custom_jwt_required()`

Questa funzione decorator si occupa di decodificare il JWT token fornito nel campo Authorization del header della richiesta mandato al backend. Se il token decodificato è valido, si estrapola le informazioni dell'utente dal database esse poi vengono immagazzinate in un ogetto di nome "g", attraverso questo oggetto possiamo accedere ad ogni campo della tabella associata all'utente che possiede il JWT. La sicurezza in questo progetto è stato un punto cruciale durante lo sviluppo, ed è per questo motivo che abbiamo scelto di implementare il JWT come si deve con un JWT SECRET in modo da evitare che un utente riesca a creare un proprio JWT, encodarlo in base64 e utilizzarlo per spacciarsi per un altro utente, ma grazie al JWT SECRET che aggiunge una firma hash alla fine del token, siamo sicuri che ogni token che arriva al server è un token valido creato dal server.

`@require_roles(*allowed)`

Questa funzione decorator viene eseguita dopo `@custom_jwt_required`, essa riceve nei parametri una lista di ruoli "ROLE_STUDENT, ROLE_REFERENT, ROLE_OVERSEAS", se l'oggetto "g" possiede un ruolo che non è presente in questa lista `*allowed` allora l'accesso viene negato e l'api restituisce errore 403 (Forbidden), se invece l'oggetto g non è inizializzato il server restituisce errore 401 (Non Autorizzato), se invece il ruolo dell'oggetto g è presente neella lista `*allowed` allora l'accesso alla funzione viene consentita.

`can_view_application(applicazione, user, role)`

Quetsa funzione invece prende in input un oggetto di tipo applicazione, ed un oggetto di tipo utente + ruolo, e verifica se l'utente ha accesso all'applicazione, se l'utente ha accesso la funzione restituisce true, altrimenti false. Il controllo all'accesso varia da ruolo a ruolo: per l'utente dev'essere vero che il suo "id" sia uguale a "user_id" dell'applicazione; per il referent dev'essere vero che il suo "id" sia uguale a referent_id dell'applicazione, mentre per lo staff dev'essere vero che il campo "id_institution" dell'utente sia uguale al campo "sending_institution" dell'applicazione.

Routes

L'applicazione flask fornisce una serie di route definite all'interno della cartella /backend/routes/*.py, ognuno dei file contiene dei route rilevanti al nome del file stesso, l'accesso ai route è controllato utilizzando le funzioni sopraelencate:

1. applications.py

- per l'inserimento, eliminazione e aggiornamento delle applicazioni
- restituire i dati associati alle applicazioni (elenco applicazioni, documenti caricati, mappature esami)
- aggiornamento dello stato dell'applicazione seguendo il flusso previsto dall'applicazione
- caricamento, download ed eliminazione dei documenti (learning agreement e transcript)
- approvazione o il rifiuto dei documenti da parte del referente
- fornire liste di valori utili al frontend (semestri, stati, anni accademici, tipi e stati dei documenti)

2. exams.py

- inserimento, eliminazione e aggiornamento degli esami o della mappatura degli esami
- restituire i dati degli esami
- aggiornamento dello stato della mappatura degli esami (approvazione e rifiuto, solo referente)
- inserimento degli esiti degli esami (voto e data di superamento)
- fornire la lista degli stati ammessi per le mappature degli esami

3. institutions.py

- inserimento, eliminazione e aggiornamento degli istituti
- inserimento, eliminazione e aggiornamento degli istituti partner (partnership)
- restituire i dati di un istituto (elenco istituti, informazioni di un singolo istituto)
- restituire le persone associate a un istituto (referenti, studenti, staff)
- fornire gli esami e gli istituti partner collegati a un istituto

4. users.py

- autenticazione dell'utente (login) e il rilascio del token di accesso
- inserimento, eliminazione e aggiornamento degli utenti (a cura dello staff)

- per restituire i dati degli utenti (elenco utenti dell'istituto, singolo utente)
- per fornire la lista dei ruoli utente ammessi

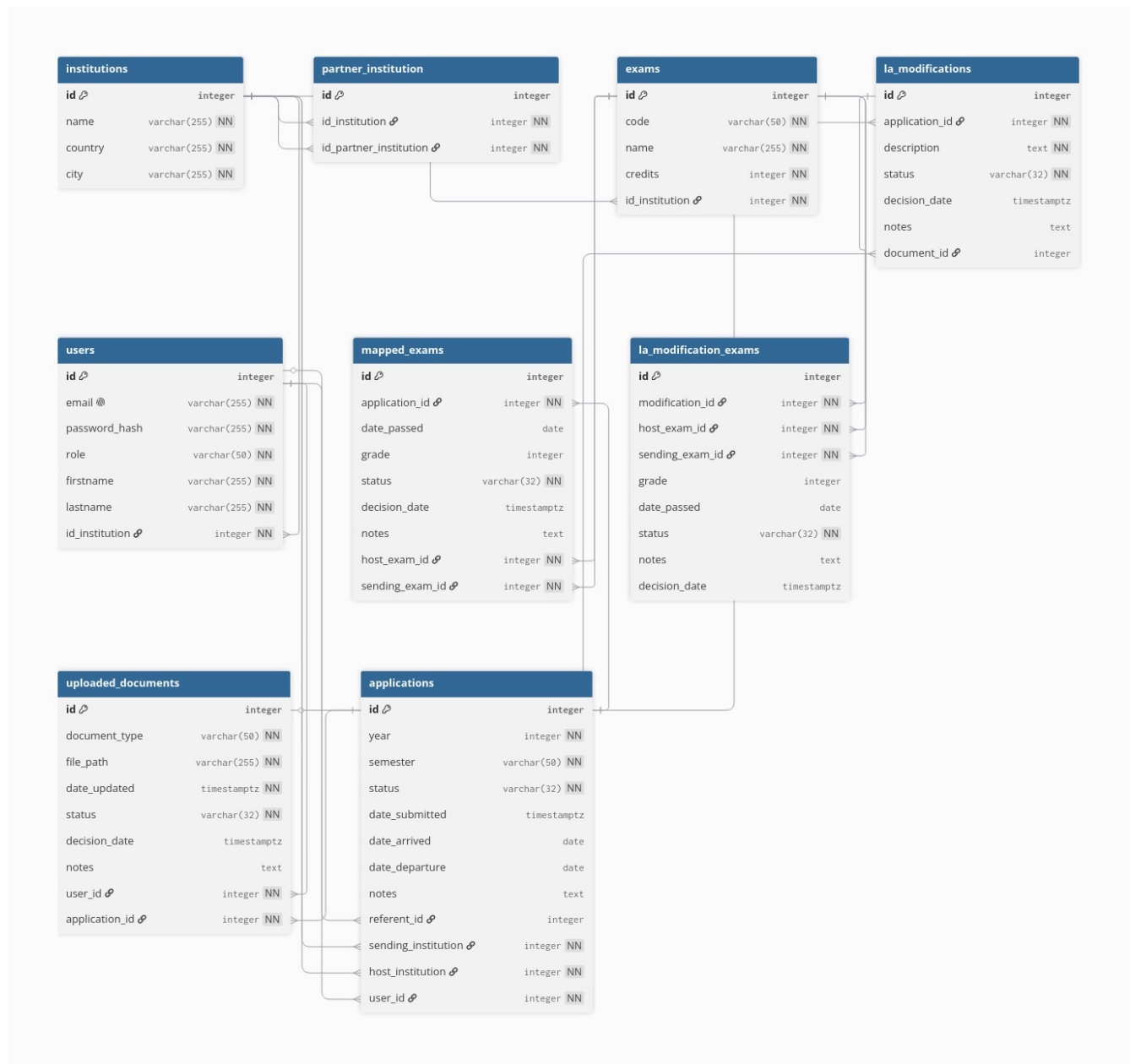
5. api.py

- verificare lo stato di salute del sistema e della connessione al database (health check)
- restituire un riepilogo generale (conteggi, ultime applicazioni, istituti con più partner)
- restituire l'elenco completo degli esami con il relativo istituto
- contiene una funzione di supporto per estrarre in modo leggibile i messaggi di errore del database

6. modifications.py

- per l'inserimento di una proposta di modifica del learning agreement da parte dello studente durante la mobilità
- restituire l'elenco delle proposte di modifica di un'applicazione con il relativo storico
- approvazione o il rifiuto di una modifica da parte del referente (con ripristino automatico della mappatura precedente in caso di rifiuto)

Progettazione Concettuale



Principali Scelte Progettuali

Politiche di integrità

Le politiche di integrità si possono trovare nel file `/docs/db/models.sql`, che abbiamo utilizzato per avere un punto di riferimento sulla struttura delle tabelle.

Per la tabella **utente** abbiamo:

- 1) opportune FK per l'istituzione di appartenenza.
- 2) assicura che il ruolo sia soltanto uno dei tre "student", "referent" o "staff"

Per la tabella **applicazioni**, vi sono una serie di vincoli di integrità tra cui:

- 1) controllo che la data di partenza non preceda la data di arrivo.
- 2) controllo che la date di arrivo e partenza siano nell'anno accademico scelto
- 3) controllo che il valore del semestre sia solo “first”, “second” o “full”
- 4) controllo che lo stato dell'applicazione sia solo uno dei seguenti: 'learning_agreement_pending', 'created', 'pre_departure_completed', 'mobility_ongoing', 'exam_recognition', 'closed'
- 5) controllo che: se lo stato dell'applicazione è in “mobility_ongoing” allora la data di arrivo (inizio mobilità) non può essere NULL.
- 6) controllo che se lo stato dell'applicazione è in “exam_recognition” allora la data di partenza (fine mobilità) non dev'essere NULL.
- 7) opportune FK per utente, referente, sending & host institutions.

Per la tabella **esami** vi sono:

- 1) controllo che *il codice dell'istituto sia univoco all'interno dello stesso istituto* (perche la tabella esami è utilizzata da tutte le istituzioni e quindi conviene accertarsi che il codice “CT3-XXXX” sia univoco per l'istituto).
- 2) Opportune FK all'istituto a cui l'esame appartiene

Per la tabella della **mappature esami** vi sono:

- 1) lo stato della mappature può essere solo uno dei 3 “approved”, “rejected”, “pending”
- 2) il voto dev'essere -1 o tra 18 e 30
- 3) se l'esame è superato (possiede un voto) la data di superamento non dev'essere nulla
- 4) lo stesso esame del sending institution non può apparire due volte per la stessa applicazione
- 5) lo stesso esame del host institution non può apparire due volte per la stessa applicazione
- 6) opportune FK per la applicazione associata e gli esami.

Per la tabella **document uploaded** abbiamo:

- 1) il tipo del documento dev'essere “learning_agreement” o “transcript”
- 2) lo stato del documento dev'essere “pending”, “approved”, “rejected”
- 3) opportune FK per l'applicazione a cui appartiene

Per la tabella **istituti partner** abbiamo:

- 1) un istituto *non può essere partner di sè stesso*
- 2) non possono esserci partnership duplicati tra istituti
- 3) opportune FK per le due istituzioni che sono partner

Per la tabella **“la modifications”** abbiamo:

- 1) lo stato della modifica può essere solo “pending”, “approved”, “rejected”
- 2) opportune FK per l'applicazione a cui si fa riferimento e anche al documento LA associato

Per la tabella **“la modification exams”** (modifiche esami, uno snapshot degli esami prima della modifica, utilizzati per ripristinare la mappature degli esami) abbiamo:

- 1) lo stesso esame di sending_institution non può apparire due volte
- 2) lo stesso esame di host_institution non può apparire due volte
- 3) opportune FK alla tabella la_modification e agli esami

Trigger Importanti

Il trigger più importante del progetto si chiama “applicaiton_status_workflow” che esegue la funzione “check_application_status_workflow()” ogni qualvolta che lo stato dell'applicazione viene cambiato. Questo trigger controlla, ad ogni passaggio di stato (status) dell'applicazione, che vi siano i requisiti fondamentali per il passaggio (learning agreement approvato, esami approvati, esami con voti e date di superamento, etc...), inoltre il trigger costringe il workflow dell'applicazione a seguire un percorso specifico:

created → learning agreement pending → created → pre departure completed → mobility ongoing → exam recognition → closed

In ognuno di questi passaggi vi sono fatti dei controlli opportuni su tutte le informazioni legate all'applicazione.

Altri trigger importanti sono:

- mapped_exam_update_status_check che controlla se il passaggio di stato della mappatura dell'esame sia lecita (con opportuni controlli)
- check_mapped_exam_institutions che controlla se la mappatura degli esami e l'applicazione associata sia valida (ovvero gli istituti presenti nell'applicazione associata combaciano con l'istituto degli esami nella mappatura)
- application_data_check controlla che i dati inseriti nell'applicazione siano inerenti, ad esempio che il referent_id sia un id che appartiene effettivamente ad un utente con ruolo “referent” (evitando che l'utente inserisca un referente che è ad esempio un suo compagno di classe).

E tanti altri trigger meno corposi che però svolgono ruoli importanti nel mantenere l'integrità e la validità dei dati nel database, visualizzare il documento in /docs/db/triggers.py

Scelte Implementative

Lo spunto di partenza richiedeva che l'applicazione fosse creata per l'utilizzo esclusivo dell'Università Ca Foscari, tuttavia abbiamo riconosciuto e ci siamo accordati che questa caratteristica era troppo limitante ed erronea a livello di usabilità e flessibilità, abbiamo dunque scelto di implementare l'intero

progetto in modo tale che qualsiasi università con i propri studenti abbia la possibilità di utilizzare il servizio, dunque, non è limitato esclusivamente alla Università di Venezia, ma è accessibile ed utilizzabile da qualsiasi istituto e tra più istituti.

Il progetto richiedeva di progettare la possibilità alle istituzioni di poter inserire ed togliere partnership con altre istituzioni, questo è possibile, abbiamo individuato che se esiste una mappatura sending → host, deve per forza esistere anche una mappatura host → sending, dunque se si inserisce una mappatura viene anche inserita una mappatura simmetrica tramite un trigger, cosicché le due istituzioni sono abilitate bilateralmente allo scambio di studenti.

L'anno accademico sono due anni (e.g. 2026/2027) abbiamo deciso di memorizzare sul database solo il primo anno come un intero, il motivo dietro questa scelta è per via del fatto che questo formato si presenta in tale modo {anno / anno+1} dunque avendo solo il primo anno possiamo ricavare anche il secondo anno aggiungendo +1, non era necessario dunque memorizzare l'anno accademico come un varchar e neanche come due interi, ma semplicemente come un intero che memorizzi l'anno.

Quando un utente crea l'applicazione è possibile che esso decida di inserire il learning agreement più in avanti prima della partenza, in questo caso l'applicazione è in stato "created", se invece è stato inserito il learning agreement l'applicazione va in stato "learning agreement pending" finché il docente referente lo approva, se viene approvato il learning agreement, lo stato dell'applicazione passa a "created". Dunque lo stato "learning agreement pending" è riservato al fatto che il learning agreement è stato inserito ma non è stato approvato, in qualsiasi altro caso lo stato è semplicemente "created" per informare l'utente che semplicemente esiste.

Per via del fatto che il progetto è condiviso con l'esame di TAW, abbiamo deciso di implementare l'intero progetto in lingua Inglese.

Ulteriori Informazioni

Ecco elencate alcune funzionalità aggiuntive **che fanno apprezzare di più il progetto**:

- Quanto l'applicazione è in stato "exam recognition", se si inserisce un voto + una data di superamento esame, il database trigger verifica che la data di superamento esame sia compresa tra la data di arrivo (inizio mobilità) e la data di partenza (fine mobilità).
- Prima che l'applicazione passi a "pre_departure_completed" viene verificata se esiste un'altra applicazione dell'utente (con stato diverso da "created" o "learning agreement pending") dove le date intersecano con le date dell'applicazione attuale, in sintesi verifica se esiste un'altra applicazione che ha superato l'approvazione e le cui date intersecano con quella che si sta cercando di far passare a pre_departure_completed.
- Per l'encoding delle password in hash abbiamo utilizzato la libreria werkzeug.security che possiede delle funzioni pre-esistenti check_password_hash e generate_password_hash per assicurarsi che le password vengono salvate nel database come delle hash e non in plaintext, per accertarsi che se avviene un database breach, almeno le password sono hashate e irrecuperabili all'attaccante.

- Quando una nuova mappatura degli esami viene inserita, vi è un trigger che controlla che gli esami combaciano con la `sending_institution` e la `host_institution` dell'applicazione associata.
- Il passaggio di stato dell'applicazione è assicurato tramite un trigger che controlla ogni passaggio e si assicura che vi siano le condizioni giuste per tale operazione.
- I membri dello staff sono abilitati all'inserimento ed eliminazione di nuovi istituti partner, sono abilitati all'inserimento, modifica e eliminazione di utenti, sono abilitati

Contributo al Progetto

LUIS SOUBIE

- Autenticazione Utente con JWT+SECRET
- Crittografia delle password nel database
- Implementato flask migration per tener traccia dei cambiamenti nel database (tabelle/modelli e trigger)
 - definizione delle tabelle:
 - `la_modifications`,
 - `la_modification_exams`
 - & trigger: `set_modification_decision_date`
- Progettazione ed implementazione delle API route per le modifiche durante “mobility ongoing”
- Implementazione del containerization Docker
- Edge Cases Testing , Bug Fixing & Security Vulnerability Assessment.

JOSEPH FRIGO

- Definizione della struttura del database
 - tabelle `users`, `application`, `institutions`, `exams`, `mapped_exams`, `uploaded_document`, `partner_institutions`
 - definizione dei trigger:
 - `applicaiton_update_on_upload`
 - `mapped_exam_institutions_check`
 - `application_data_check`
 - **`applicaiton_status_workflow`**
 - `mapped_exam_update_status_check`
 - `exam_grade_changes_check`
 - `exam_mapping_update_check`

- exam_mapping_insert_check
 - document_status_update_check
 - mirror_partner_institution
- Collegamento del backend con il frontend tramite funzioni / servizi che astraggono i route
- Implementazione del containerization Docker
- Frontend Angular Ui Templates, Components & Services.
- Definizione delle route in applications.py, users.py, exams.py, institutions.py utilizzando ORM
- Edge Cases Finding & Bug Fixing